

Generic SortedSet<T> Collection Class in C# with Examples

In this article, I am going to discuss the **Generic SortedSet<T> Collection Class in C#** with Examples. Please read our previous article where we discussed [Generic SortedList<TKey, TValue> Collection Class in C#](#) with Examples. At the end of this article, you will understand the following pointers with examples.

1. [What is SortedSet<T> in C#?](#)
2. [How to Create a Generic SortedSet Collection in C#?](#)
3. [How to Add Elements into a SortedSet Collection in C#?](#)
4. [How to Access a Generic SortedSet<T> Collection in C#?](#)
5. [How to Remove Elements from a Generic SortedSet<T> Collection in C#?](#)
6. [How to Check the Availability of an Element in a SortedSet in C#?](#)
7. [Set Operations on Generic SortedSet<T> Collection Class in C#](#)
8. [Generic SortedSet Collection with Complex Type in C#](#)
9. [How to Copy a List to a SortedSet in C#?](#)
10. [When to use SortedSet<T> Collection Class in C#?](#)

What is Generic SortedSet<T> Collection in C#?

The Generic SortedSet<T> Collection Class in C# is used to store, remove or view elements. SortedSet Collection store the elements in sorted order. That means it stores the element in ascending order and also, and it does not store duplicate elements. So, it is recommended to use the SortedSet collection if you want to store only unique elements in ascending order. This collection is of the generic type collection and hence it belongs to System.Collections.Generic namespace.

It also provides many mathematical set operations, such as intersection, union, and difference. It is a dynamic collection means the size of the SortedSet is automatically increased when new elements are added.

How to create a Generic SortedSet Collection in C#?

The Generic SortedSet Collection class in C# provided five constructors that we can use to create an instance of SortedSet. They are as follows:

1. [SortedSet\(\)](#): It initializes a new instance of the Generic SortedSet class.
2. [SortedSet\(IComparer<T> comparer\)](#): It initializes a new instance of the Generic SortedSet class that uses a specified comparer
3. [SortedSet\(IEnumerable<T> collection\)](#): It initializes a new instance of the Generic SortedSet class that contains elements copied from a specified enumerable collection.
4. [SortedSet\(IEnumerable<T> collection, IComparer<T> comparer\)](#): It initializes a new instance of the Generic SortedSet class that contains elements copied from a specified enumerable collection and that uses a specified comparer.
5. [SortedSet\(SerializationInfo info, StreamingContext context\)](#): It initializes a new instance of the Generic SortedSet class that contains serialized data. The parameter info specifies the object that contains the information that is required to serialize the Generic SortedSet object and the context parameter specifies the structure that contains the source and destination of the serialized stream associated with the Generic SortedSet object.

Let's see how to create an instance of the SortedSet using the SortedSet() constructor in C#.

Step1:

As SortedSet<T> class belongs to System.Collections.Generic namespace, so first, we need to import the System.Collections.Generic namespace into our program as follows:

```
using System.Collections.Generic;
```

Step2:

Next, we need to create an instance of the SortedSet class using the SortedSet() constructor as follows:

```
SortedSet<Type_of_SortedSet> sortedSet = new SortedSet<Type_of_SortedSet>();
```

How to Add Elements into a SortedSet Collection in C#?

If you want to add elements to your SortedSet Collection, then you need to use the following Add() method of the SortedSet class.

1. **Add(T item):** This method is used to add an element to the set and returns a value that indicates if it was successfully added. The parameter item specifies the element to add to the set. It returns true if the element is added to the SortedSet object; otherwise, false.

The following shows how to add elements using Add method of the HashSet class.

```
SortedSet<int> sortedSetNumbers = new SortedSet<int>();  
hashSetCountries.Add(10);  
hashSetCountries.Add(5);  
hashSetCountries.Add(50);
```

Even, we can also store elements in the SortedSet Collection using Collection Initializer as follows.

```
SortedSet<int> sortedSetNumbers = new SortedSet<int>  
{  
    10,  
    5,  
    50  
};
```

How to Access a Generic SortedSet<T> Collection in C#?

We can access the elements of the SortedSet<T> collection in C# using the ForEach loop as follows:

```
foreach (var item in sortedSetNumbers)  
{  
    Console.WriteLine(item);  
}
```

Even we can an enumerator to loop through the SortedSet as follows.

```
SortedSet<int>.Enumerator em = sortedSetNumbers.GetEnumerator();  
while (em.MoveNext()) {  
    int val = em.Current;  
    Console.WriteLine(val);  
}
```

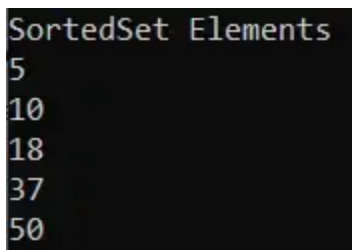
Example to Understand How to Create a SortedSet Object and Add Elements in C#:

For a better understanding of how to create a SortedSet<T> collection and how to add elements to a SortedSet, and how to access the elements of a SortedSet in C# using ForEach, please have a look at the

below example. Here, we created the HashSet collection of int type. So, the collection is going to store only integer-type values.

```
using System;
using System.Collections.Generic;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            //Creating an Instance of SortedSet class to store Integer values
            SortedSet<int> sortedSetNumbers = new SortedSet<int>();
            //Adding Elements to SortedSet using Add Method
            sortedSetNumbers.Add(10);
            sortedSetNumbers.Add(5);
            sortedSetNumbers.Add(50);
            sortedSetNumbers.Add(37);
            sortedSetNumbers.Add(18);
            sortedSetNumbers.Add(37);
            //Accessing the SortedSet Elements using For Each Loop
            Console.WriteLine("SortedSet Elements");
            foreach (var item in sortedSetNumbers)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

If you notice in the above code, we have added the numbers randomly, and also, we have added the number 37 two times. Now, if you run the above code, then you will see that numbers are stored in ascending order by removing the duplicate entry i.e. you will see 37 only once as shown in the below image.



```
SortedSet Elements
5
10
18
37
50
```

Using Enumerator to Loop through SortedSet<T> Collection in C#:

The SortedSet<T>.GetEnumerator Method is used to get an enumerator that iterates through a SortedSet object. It returns a SortedSet<T>.Enumerator object for the SortedSet<T> object. For a better understanding, please have a look at the below example. The following example will give you the same output as the previous example. Here, we are using Collection Initializer to create and initialize the SortedSet.

```
using System;
```

```

using System.Collections.Generic;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            //Creating an Instance of SortedSet and Adding Elements using Coll
Initializer SortedSet<int> sortedSetNumbers = new SortedSet<int>
        {
            10,
            5,
            50,
            37,
            18,
            37
        };
            //Accessing the SortedSet Elements using Enumerator
            Console.WriteLine("SortedSet Elements");
            SortedSet<int>.Enumerator em = sortedSetNumbers.GetEnumerator();
            while (em.MoveNext())
            {
                int val = em.Current;
                Console.WriteLine(val);
            }
            Console.ReadKey();
        }
    }
}

```

How to Remove Elements from a Generic SortedSet<T> Collection in C#?

The Generic SortedSet<T> Collection Class in C# provides the following three methods to remove elements from the HashSet.

1. **Remove(T item):** This method is used to remove the specified element from a SortedSet object. Here, the parameter item specifies the element to remove. It returns true if the element is successfully found and removed; otherwise, false. This method returns false if the item is not found in the Generic SortedSe collection.
2. **RemoveWhere(Predicate<T> match):** This method is used to remove all elements that match the conditions defined by the specified predicate from a SortedSet collection. It returns the number of elements that were removed from the SortedSet collection. Here, the parameter match specifies the Predicate delegate that defines the conditions of the elements to remove.
3. **Clear():** This method is used to remove all elements from a SortedSet object.

Let us see an example to understand the above three methods of Generic SortedSet Collection Class in C#. Please have a look at the below example where we created a SortedSet of string types.

```

using System;

```

```

using System.Collections.Generic;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            //Creating SortedSet and Adding Elements to SortedSet using Collec
Initializer SortedSet<string> sortedSetCountries = new SortedSet<string>()
        {
            "BANGLADESH",
            "NEPAL"
        };
        //Adding Elements to SortedSet using Add Method
sortedSetCountries.Add("INDIA");
sortedSetCountries.Add("USA");
sortedSetCountries.Add("UK");
        Console.WriteLine($"SortedSet Elements Count Before Removing:
{sortedSetCountries.Count}");
        foreach (var item in sortedSetCountries)
        {
            Console.WriteLine(item);
        }
        // Remove element Bangladesh from SortedSet Using Remove() method
sortedSetCountries.Remove("Bangladesh");
        Console.WriteLine($"SortedSet Elements Count After Removing Bang
{sortedSetCountries.Count}");
        foreach (var item in sortedSetCountries)
        {
            Console.WriteLine(item);
        }
        // Remove Element from SortedSet Using RemoveWhere() method where
length is > 3
sortedSetCountries.RemoveWhere(x => x.Length > 3);
        Console.WriteLine($"SortedSet Elements Count After Removeing Ele
Length > 3: {sortedSetCountries.Count}");
        foreach (var item in sortedSetCountries)
        {
            Console.WriteLine(item);
        }
        // Remove all Elements from SortedSet Using Clear method
sortedSetCountries.Clear();
        Console.WriteLine($"SortedSet Elements Count After Clear:
{sortedSetCountries.Count}");
        Console.ReadKey();
    }
}

```

Output:

```
SortedSet Elements Count Before Removing: 5
BANGLADESH
INDIA
NEPAL
UK
USA

SortedSet Elements Count After Removing Bangladesh: 5
BANGLADESH
INDIA
NEPAL
UK
USA

SortedSet Elements Count After Removeing Elements whose Length > 3: 2
UK
USA

SortedSet Elements Count After Clear: 0
```

How to Check the Availability of an Element in a SortedSet in C#?

If you want to check whether an element exists or not in the SortedSet, then you can use the following Contains() method of the Generic SortedSet Collection Class in C#.

1. **Contains(T item):** This method is used to determine whether a SortedSet object contains the specified element. The parameter item specifies the element to locate in the SortedSet object. It returns true if the SortedSet object contains the specified element; otherwise, false.

Let us understand this with an example. The following example shows how to use the Contains() method of the Generic SortedSet Collection class in C#.

```
using System;
using System.Collections.Generic;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            SortedSet<string> sortedSetCountries = new SortedSet<string>();
            //Adding Elements to SortedSet using Add Method
            sortedSetCountries.Add("INDIA");
            sortedSetCountries.Add("USA");
            sortedSetCountries.Add("UK");
            //Checking the key using the Contains method
```

```

        Console.WriteLine("Is INDIA Key Exists : " +
sortedSetCountries.Contains("INDIA"));
        Console.WriteLine("Is NZ Key Exists : " + sortedSetCountries.Conta
        Console.ReadKey();
    }
}
}

```

Output:

```

Is INDIA Key Exists : True
Is NZ Key Exists : False

```

Set Operations on Generic SortedSet<T> Collection Class in C#

The Generic SortedSet Collection Class in C# also provides some methods that we can use to perform different set operations. The methods are as follows.

1. **UnionWith(IEnumerable<T> other):** This method is used to modify the current SortedSet object to contain all elements that are present in itself, the specified collection, or both. Here, the parameter other specifies the collection to compare to the current SortedSet object. If the parameter other is null, then we will get ArgumentNullException.
2. **IntersectWith(IEnumerable<T> other):** This method is used to modify the current SortedSet object to contain only elements that are present in that object and in the specified collection. Here, the parameter other specifies the collection to compare to the current SortedSet object. If the parameter other is null, then we will get ArgumentNullException.
3. **ExceptWith(IEnumerable<T> other):** This method is used to remove all elements in the specified collection from the current SortedSet object. Here, the parameter other specifies the collection of items to remove from the SortedSet object. If the parameter other is null, then we will get ArgumentNullException.
4. **SymmetricExceptWith(IEnumerable<T> other):** This method is used to modify the current SortedSet object to contain only elements that are present either in that object or in the specified collection, but not both. Here, the parameter other specifies the collection to compare to the current SortedSet object. If the parameter other is null, then it will throw ArgumentNullException.

Generic SortedSet Collection UnionWith(IEnumerable<T> other) Example in C#:

This method is used to modify the current SortedSet object to contain all elements that are present in itself, the specified collection, or both. For a better understanding please have a look at the below example where we created a SortedSet collection object of string type. Here, you will see that the UnionWith method will contain the elements which are present in both the collection by removing the duplicate elements.

```

using System;
using System.Collections.Generic;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {

```

```

//Creating SortedSet 1
SortedSet<string> sortedSetCountries1 = new SortedSet<string>();
//Adding Elements to sortedSetCountries1 using Add Method
sortedSetCountries1.Add("IND");
sortedSetCountries1.Add("USA");
sortedSetCountries1.Add("UK");
sortedSetCountries1.Add("NZ");
sortedSetCountries1.Add("BAN");
Console.WriteLine("SortedSet 1 Elements");
foreach (var item in sortedSetCountries1)
{
    Console.WriteLine(item);
}
//Creating SortedSet 2
SortedSet<string> sortedSetCountries2 = new SortedSet<string>();
//Adding Elements to HashSet using Add Method
sortedSetCountries2.Add("IND");
sortedSetCountries2.Add("SA");
sortedSetCountries2.Add("PAK");
sortedSetCountries2.Add("USA");
sortedSetCountries2.Add("ZIM");
Console.WriteLine("\nSortedSet 2 Elements");
foreach (var item in sortedSetCountries2)
{
    Console.WriteLine(item);
}
// Using UnionWith method
sortedSetCountries1.UnionWith(sortedSetCountries2);
Console.WriteLine("\nSortedSet 1 Elements After UnionWith");
foreach (var item in sortedSetCountries1)
{
    Console.WriteLine(item);
}
Console.ReadKey();
}
}
}

```

Output:

```
SortedSet 1 Elements
BAN
IND
NZ
UK
USA

SortedSet 2 Elements
IND
PAK
SA
USA
ZIM

SortedSet 1 Elements After UnionWith
BAN
IND
NZ
PAK
SA
UK
USA
ZIM
```

Generic SortedSet Collection IntersectWith(IEnumerable<T> other) Example in C#:

This method is used to modify the current SortedSet object to contain only elements that are present in that object and in the specified collection. For a better understanding please have a look at the below example where we created a SortedSet collection object of string type. Here, you will see that the IntersectWith method will contain the common elements which are present in both the collection.

```
using System;
using System.Collections.Generic;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            //Creating SortedSet 1
            SortedSet<string> sortedSetCountries1 = new SortedSet<string>();
            //Adding Elements to sortedSetCountries1 using Add Method
            sortedSetCountries1.Add("IND");
            sortedSetCountries1.Add("USA");
            sortedSetCountries1.Add("UK");
            sortedSetCountries1.Add("NZ");
            sortedSetCountries1.Add("BAN");
            Console.WriteLine("SortedSet 1 Elements");
            foreach (var item in sortedSetCountries1)
```

```

    {
        Console.WriteLine(item);
    }
    //Creating SortedSet 2
    SortedSet<string> sortedSetCountries2 = new SortedSet<string>();
    //Adding Elements to HashSet using Add Method
    sortedSetCountries2.Add("IND");
    sortedSetCountries2.Add("SA");
    sortedSetCountries2.Add("PAK");
    sortedSetCountries2.Add("USA");
    sortedSetCountries2.Add("ZIM");
    Console.WriteLine("\nSortedSet 2 Elements");
    foreach (var item in sortedSetCountries2)
    {
        Console.WriteLine(item);
    }
    // Using IntersectWith method
    sortedSetCountries1.IntersectWith(sortedSetCountries2);
    Console.WriteLine("\nSortedSet 1 Elements After IntersectWith");
    foreach (var item in sortedSetCountries1)
    {
        Console.WriteLine(item);
    }
    Console.ReadKey();
}
}
}

```

Output:

```

SortedSet 1 Elements
BAN
IND
NZ
UK
USA

SortedSet 2 Elements
IND
PAK
SA
USA
ZIM

SortedSet 1 Elements After IntersectWith
IND
USA

```

Generic SortedSet Collection ExceptWith(IEnumerable<T> other) Example in C#:

This method is used to remove all elements in the specified collection from the current SortedSet object. For a better understanding please have a look at the below example where we created a SortedSet collection object of string type. Here, you will see that the ExceptWith method will contain the elements from the first collection which are not present in the second collection.

```
using System;
using System.Collections.Generic;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            //Creating SortedSet 1
            SortedSet<string> sortedSetCountries1 = new SortedSet<string>();
            //Adding Elements to sortedSetCountries1 using Add Method
            sortedSetCountries1.Add("IND");
            sortedSetCountries1.Add("USA");
            sortedSetCountries1.Add("UK");
            sortedSetCountries1.Add("NZ");
            sortedSetCountries1.Add("BAN");
            Console.WriteLine("SortedSet 1 Elements");
            foreach (var item in sortedSetCountries1)
            {
                Console.WriteLine(item);
            }
            //Creating SortedSet 2
            SortedSet<string> sortedSetCountries2 = new SortedSet<string>();
            //Adding Elements to HashSet using Add Method
            sortedSetCountries2.Add("IND");
            sortedSetCountries2.Add("SA");
            sortedSetCountries2.Add("PAK");
            sortedSetCountries2.Add("USA");
            sortedSetCountries2.Add("ZIM");
            Console.WriteLine("\nSortedSet 2 Elements");
            foreach (var item in sortedSetCountries2)
            {
                Console.WriteLine(item);
            }
            // Using ExceptWith method
            sortedSetCountries1.ExceptWith(sortedSetCountries2);
            Console.WriteLine("\nSortedSet 1 Elements After ExceptWith ");
            foreach (var item in sortedSetCountries1)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
SortedSet 1 Elements
BAN
IND
NZ
UK
USA

SortedSet 2 Elements
IND
PAK
SA
USA
ZIM

SortedSet 1 Elements After ExceptWith
BAN
NZ
UK
```

Generic SortedSet Collection SymmetricExceptWith(IEnumerable<T> other) Example in C#:

This method is used to modify the current SortedSet object to contain only elements that are present either in that object or in the specified collection, but not both. For a better understanding please have a look at the below example where we created a SortedSet collection of string types. Here, you will see that the SymmetricExceptWith method will contain the elements which are not common in both collections.

```
using System;
using System.Collections.Generic;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            //Creating SortedSet 1
            SortedSet<string> sortedSetCountries1 = new SortedSet<string>();
            //Adding Elements to sortedSetCountries1 using Add Method
            sortedSetCountries1.Add("IND");
            sortedSetCountries1.Add("USA");
            sortedSetCountries1.Add("UK");
            sortedSetCountries1.Add("NZ");
            sortedSetCountries1.Add("BAN");
            Console.WriteLine("SortedSet 1 Elements");
            foreach (var item in sortedSetCountries1)
            {
```

```

        Console.WriteLine(item);
    }
    //Creating SortedSet 2
    SortedSet<string> sortedSetCountries2 = new SortedSet<string>();
    //Adding Elements to HashSet using Add Method
    sortedSetCountries2.Add("IND");
    sortedSetCountries2.Add("SA");
    sortedSetCountries2.Add("PAK");
    sortedSetCountries2.Add("USA");
    sortedSetCountries2.Add("ZIM");
    Console.WriteLine("\nSortedSet 2 Elements");
    foreach (var item in sortedSetCountries2)
    {
        Console.WriteLine(item);
    }
    // Using ExceptWith method
    sortedSetCountries1.SymmetricExceptWith(sortedSetCountries2);
    Console.WriteLine("\nSortedSet 1 Elements After SymmetricExceptWith");
    foreach (var item in sortedSetCountries1)
    {
        Console.WriteLine(item);
    }
    Console.ReadKey();
}
}
}

```

Output:

```
SortedSet 1 Elements
BAN
IND
NZ
UK
USA

SortedSet 2 Elements
IND
PAK
SA
USA
ZIM

SortedSet 1 Elements After SymmetricExceptWith
BAN
NZ
PAK
SA
UK
ZIM
```

Generic SortedSet Collection with Complex Type in C#:

As of now, we have used the built-in string and integer type with SortedSet. Now, let us see how to create a SortedSet collection of Complex types i.e. user-defined class types. Let us create a class called Student and then create a SortedSet collection of Student types and also add some duplicate elements. For a better understanding, please have a look at the below example.

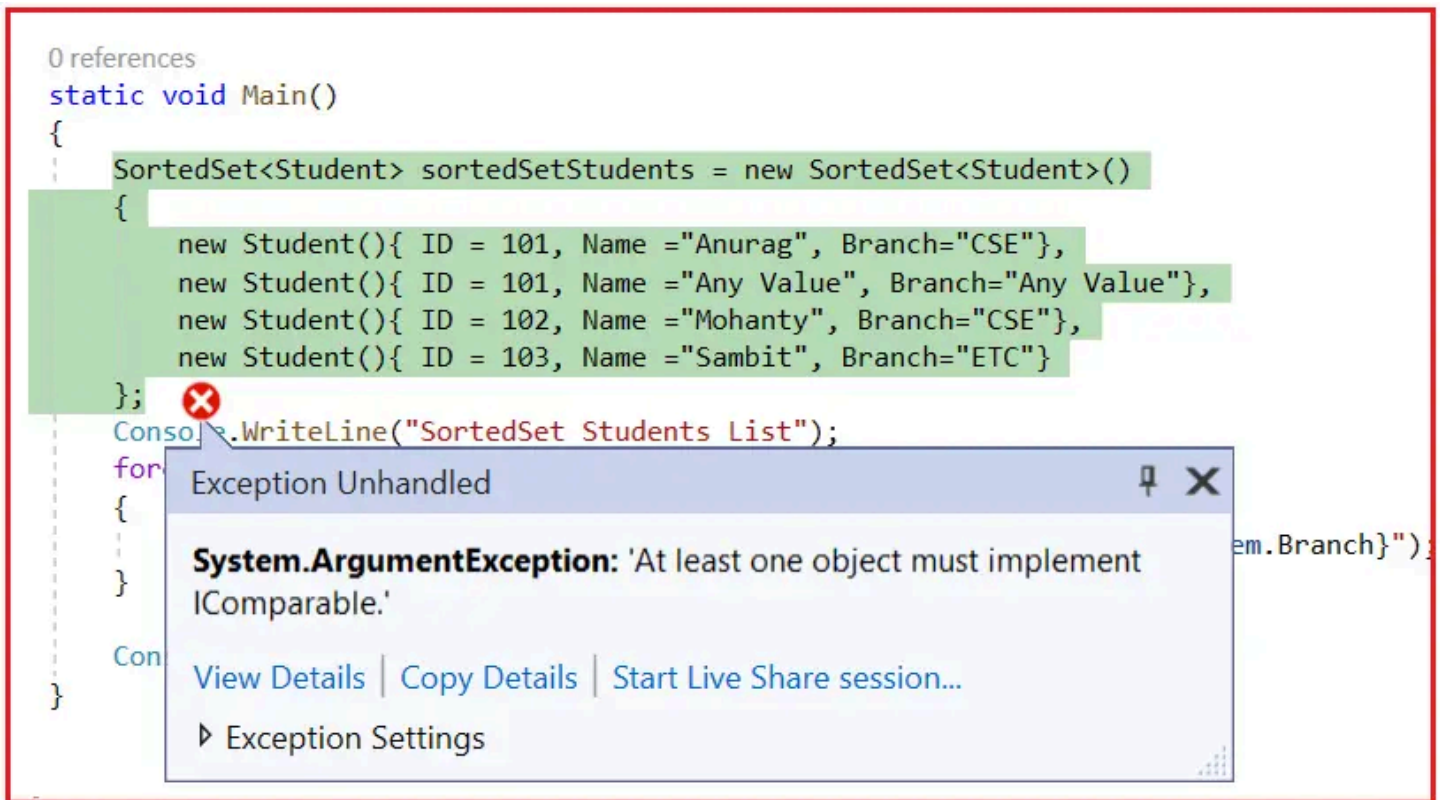
```
using System;
using System.Collections.Generic;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            SortedSet<Student> sortedSetStudents = new SortedSet<Student>()
            {
                new Student(){ ID = 101, Name = "Anurag", Branch="CSE"},
                new Student(){ ID = 101, Name = "Any Value", Branch="Any Value"},
                new Student(){ ID = 102, Name = "Mohanty", Branch="CSE"},
                new Student(){ ID = 103, Name = "Sambit", Branch="ETC"}
            };
            Console.WriteLine("SortedSet Students List");
            foreach (var item in sortedSetStudents)
            {
                Console.WriteLine($"ID: {item.ID}, Name: {item.Name}, Branch:
{item.Branch}");
            }
        }
    }
}
```

```

    }
    Console.ReadKey();
}
}
public class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
}
}

```

Now, when you run the above code, you will get the following exception.



This is because the SortedSet is unable to identify how to sort the data for students. So, we should tell how to sort the elements by implementing the IComparable interface and providing an implementation for the CompareTo method. So, in our example, the Student class should implement the IComparable<Student> interface and provide an implementation for the CompareTo method as shown in the below example. Here, we are comparing based on the ID column values.

```

using System;
using System.Collections.Generic;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {

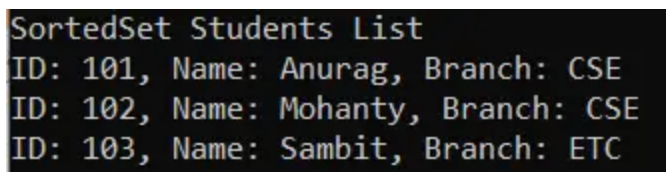
```

```

SortedSet<Student> sortedSetStudents = new SortedSet<Student>()
{
    new Student(){ ID = 101, Name = "Anurag", Branch="CSE"},
    new Student(){ ID = 101, Name = "Any Value", Branch="Any Value"},
    new Student(){ ID = 102, Name = "Mohanty", Branch="CSE"},
    new Student(){ ID = 103, Name = "Sambit", Branch="ETC"}
};
Console.WriteLine("SortedSet Students List");
foreach (var item in sortedSetStudents)
{
    Console.WriteLine($"ID: {item.ID}, Name: {item.Name}, Branch:
{item.Branch}");
}
Console.ReadKey();
}
}
public class Student : IComparable<Student>
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
    public int CompareTo(Student other)
    {
        if (this.ID > other.ID)
        {
            return 1;
        }
        else if (this.ID < other.ID)
        {
            return -1;
        }
        else
        {
            return 0;
        }
    }
}
}
}

```

Now, run the above code, and you will get the output as expected as shown in the below image.



```

SortedSet Students List
ID: 101, Name: Anurag, Branch: CSE
ID: 102, Name: Mohanty, Branch: CSE
ID: 103, Name: Sambit, Branch: ETC

```

Now, you may have one query. Why we are getting this error in our user-defined class? Why we are not getting the error in built-in data types? The answer is the built-in data type already implemented the IComparable interface and hence we are not getting the error. If you go to the definition of any built-in data

type such as int, then you will see the Int32 struct already implement the IComparable interface as shown below.

```
//  
// Summary:  
// Represents a 32-bit signed integer.  
public readonly struct Int32 : IComparable, IComparable<Int32>, IConvertible, IEquatable<Int32>, IFormattable  
{
```

How to copy a list to a SortedSet in C#?

To copy a list to a SortedSet, we need to use the following overloaded constructor of the SortedSet class. This constructor takes one parameter of IEnumerable<T>. As we know the List<T> implements IEnumerable<T>, so we can pass a List<T> collection as a parameter to the following constructor.

SortedSet(IEnumerable<T> collection);

For a better understanding, please have a look at the below example. Here, first, we created a string list to store countries, and then we created one SortedList collection object by passing the string list as a parameter to the constructor.

```
using System;  
using System.Collections.Generic;  
namespace GenericsDemo  
{  
    class Program  
    {  
        static void Main()  
        {  
            List<string> listCountries = new List<string>()  
            {  
                "INDIA",  
                "USA",  
                "UK"  
            };  
            SortedSet<string> sortedSetCountries = new SortedSet<string>(listCountries);  
            foreach (var item in sortedSetCountries)  
            {  
                Console.WriteLine($"{item}");  
            }  
            Console.ReadKey();  
        }  
    }  
}
```

Generic SortedSet Collection Class Properties in C#

Following are the properties provided by the SortedSet<T> Class.

1. **Min**: Returns the minimum value in the set
2. **Max**: Returns the maximum value in the set
3. **Count**: Returns the number of elements in the SortedSet.
4. **Comparer**: Returns the comparer that is used to order the values in the Generic SortedSet.

Example to Understand Generic SortedSet Collection Class Properties in C#

```
using System;
using System.Collections.Generic;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            SortedSet<int> sortedSetNumbers = new SortedSet<int>
            {
                10,
                5,
                50,
                37,
                18
            };
            Console.WriteLine("SortedSet Elements");
            foreach (var item in sortedSetNumbers)
            {
                Console.WriteLine(item);
            }
            Console.WriteLine($"Min Property : {sortedSetNumbers.Min}");
            Console.WriteLine($"Max Property : {sortedSetNumbers.Max}");
            Console.WriteLine($"Count Property : {sortedSetNumbers.Count}");
            Console.WriteLine($"Comparer Property : {sortedSetNumbers.Comparer}");
            Console.ReadKey();
        }
    }
}
```

Output:

```
SortedSet Elements
5
10
18
37
50
Min Property : 5
Max Property : 50
Count Property : 5
Comparer Property : System.Collections.Generic.GenericComparer`1[System.Int32]
```

When to use SortedSet<T> Collection Class in C#?

We need to use Generic SortedSet<T> Collection if we want to store unique elements and maintain ascending order.

Note: A SortedSet<T> object maintains a sorted order without affecting performance as elements are inserted and deleted. Duplicate elements are not allowed. Changing the sort values of existing items is not supported and may lead to unexpected behavior.

Summary of Generic SortedSet<T> Collection Class in C#:

1. The Generic SortedSet<T> Collection Class implements the ICollection<T>, IEnumerable<T>, IEnumerable, IReadOnlyCollection<T>, ISet<T>, ICollection, IDeserializationCallback, ISerializable interfaces.
2. The capacity of a SortedSet<T> collection is the number of elements it can hold.
3. The Generic SortedSet<T> Collection provides many mathematical set operations, such as intersection, union, and difference.
4. It does not allow the addition of duplicate elements i.e. the elements must be unique in SortedSet<T>.
5. In SortedSet, the order of the element is ascending.
6. The Generic SortedSet<T> in C# is a dynamic collection. That means the size of the SortedSet is automatically increased when new elements are added to the collection.
7. As the SortedSet<T> is a Generic Collection, so we can only store the same type of elements.

In the next article, I am going to discuss the [Generic SortedDictionary<TKey, TValue> Collection Class in C#](#) with Examples. Here, in this article, I try to explain the **Generic SortedSet<T> Collection Class in C#** with Examples. I hope this Generic SortedSet<T> Collection Class in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.